



Bloom: A Default-Deny Risk Lattice and Consensus-Enforced Agent Authority for Tokenized Equities

Project Bloom

Arbitrum Open House Singapore: Online Buildathon 2026

September 25, 2026 · Robinhood Chain Testnet (chain 46630) · Stylus engine 0xc464...b124

Source: github.com/Aaditya1273/BL00M · tag `bloom-v2-rc1`

Abstract

Tokenized stocks trade around the clock; the equities behind them do not. They halt, split, re-base, and fall silent after hours, and the oracle that prices them pauses precisely when those events land. A lending protocol or an autonomous agent that treats a fresh price as permission to act is therefore wrong in exactly the moments that matter. We present *Bloom*, a consumer wallet on Robinhood Chain whose every collateral and agent decision is gated by an equity-aware risk engine running as an Arbitrum Stylus (Rust/WASM) contract. The engine is a *default-deny lattice*: eight states evaluated as ordered guards, where the single state that permits borrowing is reachable only when every adverse predicate is false, and a silent reporter can only ever move an asset away from it. Market status (halts, corporate actions, reference prices) enters through EIP-712 signed reports bound to chain and contract, with strictly monotonic nonces and bounded age. Agent authority is not a prompt: a goal written in plain English compiles to an onchain policy (closed call set, asset allowlist, per-transaction and daily caps, expiry) that an ERC-4337 smart account enforces, so no off-chain component, including the model, the backend, or a compromised server, can widen what the agent may do. We formalize the lattice and the authorization predicate, give an authority analysis of every component, and evaluate against the live deployment: two independent implementations (Rust/Stylus and Solidity) agree on 43 shared specification vectors; 197 automated tests pass; the full demonstration passes 13/13 checks against the live testnet. We also report a result that contradicts the usual expectation: for this call-bound workload the Stylus engine costs $1.57\times$ the gas of its EVM twin to ingest a report and $1.95\times$ to evaluate risk, and we explain why. Every number in this paper was measured.

Keywords: tokenized equities; oracle risk; Arbitrum Stylus; default-deny state machines; EIP-712; ERC-4337 session keys; autonomous agents; policy enforcement; differential testing.

1 Introduction

A price feed answers one question: *what is the last traded price?* Every lending market built on it silently answers a second one, *is it safe to lend against this asset right now?*, by assuming the answer is “yes whenever the price is fresh.” For crypto-native collateral that assumption is mostly harmless. For

a tokenized equity it is wrong in a structured, predictable way. The underlying stock can be halted by its exchange while the token keeps trading; a 10:1 split re-bases every price and balance at once; the price oracle is explicitly paused while the corporate action lands; and outside the regular session, most of every day, the last print is simply old. A protocol reading only the feed sees one confident number through all of it, and keeps lending and liquidating as if nothing had happened.

A second shift compounds the first. Wallets are acquiring agents: software that turns “save for my laptop” into transactions. In most agent systems the only thing standing between a model and a user’s money is a system prompt. The objections are the familiar ones, since a model is injectable, nondeterministic and confident when wrong, but in finance the failure is not a bad answer: it is a transfer.

Bloom’s thesis is that **a price is not a permission**. Whether collateral may be borrowed against, whether a position may be liquidated, and whether an agent may touch an asset are separate decisions that must be *interpreted* from market status and *enforced* below the application layer, where neither a model nor a compromised server can reach them. Concretely, Bloom contributes:

- **A default-deny risk lattice** for tokenized equities (Section 5), specified once, implemented twice (Rust on Arbitrum Stylus and Solidity), and checked against 43 shared vectors. Borrowing is permitted in exactly one of eight states; the others collapse maximum loan-to-value to zero and put liquidations into a protected mode.
- **Authenticated market status** (Section 6): EIP-712 reports carrying halt, corporate-action and reference-price signals, accepted only with strictly increasing nonces, bounded age, in-order time-stamps and low-s signatures from an allowlisted reporter, with the property that reporter *silence* can only deny.
- **Consensus-enforced agent authority** (Section 7): plain-English goals compile to an onchain BloomPolicy; an ERC-4337 account accepts agent operations only through one entry point that consults it. The model is upstream of every check and can only lose authority, never gain it, a property that holds even if the Bloom backend is compromised.
- **An authority analysis** (Table 2) enumerating, for every key and component, which outcomes it can and cannot cause.
- **A measured evaluation on the live deployment** (Section 10), including a Stylus-versus-EVM gas comparison that goes against the expectation, the operating cost of the reporter, and thirteen defects found only by testing against live services (Section 11).

The claim is deliberately narrow. Bloom governs *collateral and agent decisions*, not market prices: it does not replace the oracle, it interprets it. It trusts an allowlisted reporter for the truth of market status (Section 13). On testnet the stock tokens, USDG and price feeds are mocks that relay live Robinhood API quotes; the risk engine and all enforcement are real contracts.

2 Background

Robinhood Chain is an Arbitrum Orbit chain (testnet 46630, mainnet 4663) whose native assets include Robinhood Stock Tokens (tokenized instruments giving economic exposure to US equities; availability is jurisdiction-dependent), canonical USDG, Chainlink Stock Token price feeds, and the canonical ERC-4337 v0.8 EntryPoint. **Arbitrum Stylus** executes WASM contracts compiled from Rust alongside the EVM with a shared state and ABI; the chain reports `ArbWasm.stylusVersion() = 3`. **Chainlink feeds** expose `latestRoundData()` with an update time and a heartbeat after which a value is stale; the stock feeds carry 8 decimals, a 24-hour heartbeat and include the corporate-action multiplier. An **L2 sequencer uptime feed** reports sequencer liveness; none is published for

Robinhood Chain, a fact Bloom records explicitly onchain rather than faking. **ERC-4626** standardizes yield vaults; **ERC-4337** standardizes smart accounts whose `validateUserOp` decides which signatures authorize which calls.

3 Threat Model and Design Goals

We consider eight principals. The **user** owns a BloomAccount and is trusted over it. The **model** (an optional language model behind the chat) is untrusted: it may be injected, hallucinate, or emit malformed output. The **backend** holds the agent session key and relays actions; it is trusted for availability, not for authority. The **reporter** is trusted for the truth of market status within bounds (Section 6). The **admin** (intended to be a multisig) configures assets and reporters; the **guardian** may only pause. **Oracles** and the **stock-token issuer** are trusted for prices, multipliers and pause flags, but not for liveness. **Everyone else**, including other contracts, can call any public function.

Goals. (G1) *Default deny*: an asset is borrowable only if every risk predicate affirmatively holds; any missing, malformed, stale or contradictory input denies. (G2) *Authority monotonicity*: no component downstream of the model, and no off-chain component at all, can widen an agent’s authority beyond the user’s onchain policy. (G3) *Silence denies*: the failure of any data source (reporter, oracle, sequencer, issuer) can remove permissions but never grant them. (G4) *Protected liquidation*: no position is liquidated while any of its collateral is outside the permitting state. (G5) *Verifiable parity*: the risk logic has one specification and two implementations that must agree on it. (G6) *Honest degradation*: every refusal carries a machine-readable reason and a plain-English explanation.

Non-goals. Detecting a *false* report from an honest-looking reporter (the reporter is a trust assumption, mitigated by rotation and bounds, not eliminated); governing the model’s reasoning (Bloom governs its effects); price discovery; and availability under a total RPC outage (actions fail closed, Section 11).

4 System Architecture

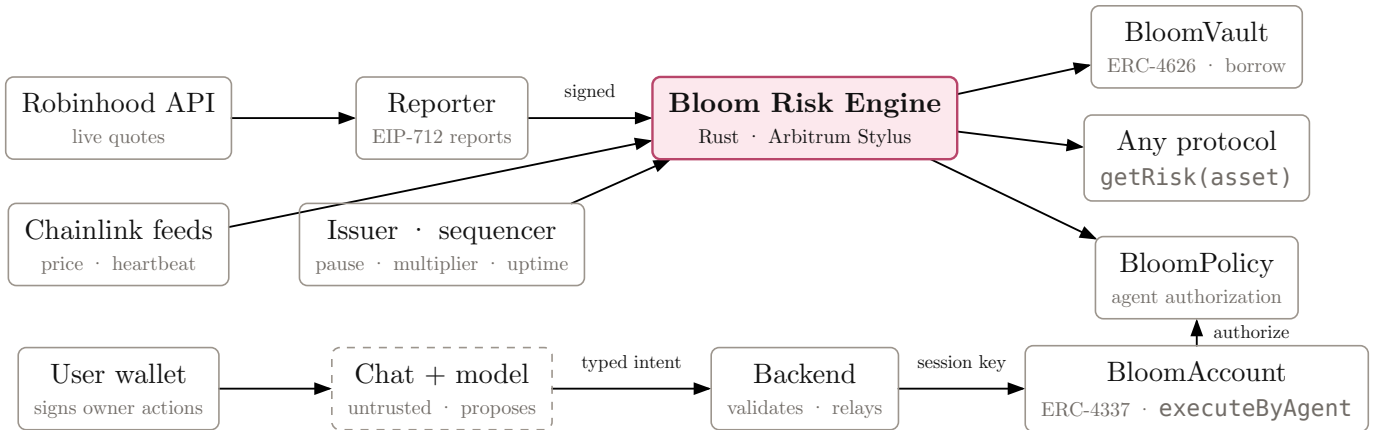


Figure 1: Bloom architecture. The shaded engine is the sole source of risk truth; the vault, the agent policy and any third-party protocol consult it. The model (dashed) is untrusted and can only propose typed intents; authority is decided onchain by **BloomPolicy** and enforced by the account.

Bloom has two flows (Figure 1). In the **risk flow**, the reporter fetches quotes and market status from the Robinhood API, signs EIP-712 reports, and submits them to the engine, which combines them with the Chainlink price, the token’s own pause flag and multiplier, and sequencer uptime. In the **action flow**,

the user speaks to the chat; a deterministic parser (with an optional model for unrecognized phrasing, whose output passes the same schema validator) produces a typed intent; owner actions return to the user’s own wallet as sign requests (the backend never holds user keys); agent actions are submitted with a goal-scoped session key through `BloomAccount.executeByAgent`, which calls `BloomPolicy.authorize` for every inner call.

The implementation comprises 1,766 lines of Solidity (plus 367 of clearly labelled testnet mocks), 1,401 lines of Rust for the Stylus engine, a 2,020-line TypeScript backend, a 577-line reporter, and a 4,919-line Next.js frontend.

5 The Risk Lattice

5.1 Inputs and predicates

For an asset a at time t the engine reads: its configuration (configured flag, heartbeat $h > 0$, deviation bound $\delta \in (0, 5000]$ basis points, maximum LTV $L_a \leq 8000$ bps, whether it is a stock token); the feed round $(r_{\text{id}}, p_{\text{raw}}, u, r_{\text{ans}})$ with decimals d ; the latest accepted report (halted flag η , corporate-action flag κ , multiplier m_r , reference price ρ , observation time o); the token’s `oraclePaused()` flag π and multiplier m_t ; and the sequencer answer, start time s and grace period γ . Every external read is a static call whose failure is converted into a *not-ok* input rather than a revert, so a broken dependency is a signal, not an outage. Prices are normalized to 18 decimals and bounded:

$$\text{norm}(p_{\text{raw}}, d) = \begin{cases} p_{\text{raw}} \cdot 10^{18-d} & \text{if } 0 < p_{\text{raw}} \leq P_{\text{max}} \text{ and } d \leq 18 \\ \lfloor p_{\text{raw}} / 10^{d-18} \rfloor & \text{if } 0 < p_{\text{raw}} \leq P_{\text{max}} \text{ and } 18 < d \leq 36 \\ \perp & \text{otherwise} \end{cases} \quad (1)$$

with $P_{\text{max}} = 10^{36}$; a result of zero or above P_{max} is also $\perp$. Seven adverse predicates follow:

Predicate	Holds when (any disjunct)
U unsupported	asset not configured
Q sequencer	sequencer required and (read failed $\vee$ answer $\neq 0 \vee s = 0 \vee s > t \vee t - s \leq \gamma$)
I invalid price	feed read failed $\vee u = 0 \vee u > t \vee r_{\text{ans}} < r_{\text{id}} \vee \text{norm}(p_{\text{raw}}, d) = \perp$
S stale	$t - u > h \vee$ (stock token and (no report $\vee o > t \vee t - o > A$))
H halted	stock token and η
C corporate action	stock token and (token read failed $\vee \pi \vee \kappa \vee (m_r \neq 0 \text{ and } m_r \neq m_t)$)
D deviation	$\rho \neq 0$ and $(\rho > P_{\text{max}} \vee p - \rho \cdot 10^4 > \delta \cdot \rho)$

Table 1: The seven adverse predicates. A is the maximum report age (3,600 s on testnet); p is the normalized price. The deviation test is an exact integer inequality, with both sides bounded by 10^{40} , so no division or rounding occurs.

5.2 Classification as ordered guards

The state is the first guard that fires, in a fixed order; if none fires, the asset is NORMAL:

$$\sigma(a, t) = \begin{cases} \text{UNSUPPORTED} & \text{if } U \\ \text{SEQUENCER_DOWN} & \text{else if } Q \\ \text{INVALID_PRICE} & \text{else if } I \\ \text{STALE} & \text{else if } S \\ \text{HALTED} & \text{else if } H \\ \text{CORP_ACTION_PAUSED} & \text{else if } C \\ \text{DEVIATION} & \text{else if } D \\ \text{NORMAL} & \text{otherwise.} \end{cases} \quad (2)$$

The order is deliberate: facts about the *data* (is there a configured asset, a live sequencer, a valid price, a fresh report) are established before facts about the *market* (halted, corporate action, deviation), so that a market signal is never interpreted from untrustworthy data. The output is

$$\text{LTV}(a) = \begin{cases} L_a & \text{if } \sigma(a, t) = \text{NORMAL} \\ 0 & \text{otherwise} \end{cases}, \quad \text{borrow}(a) = \text{liquidate}(a) = [\sigma(a, t) = \text{NORMAL}], \quad (3)$$

and the price is published only when $\sigma \notin \{\text{UNSUPPORTED}, \text{SEQUENCER_DOWN}, \text{INVALID_PRICE}\}$, so a consumer can display a stale or halted price but never obtains one derived from invalid data.

5.3 Properties

Property 1 (default deny). $\sigma(a, t) = \text{NORMAL} \Leftrightarrow \neg U \wedge \neg Q \wedge \neg I \wedge \neg S \wedge \neg H \wedge \neg C \wedge \neg D$. *Proof.* Immediate from Equation 2: NORMAL is reached only through the final branch, which requires every guard to be false; conversely, if all predicates are false no earlier branch fires. $\square$

Property 2 (adverse monotonicity). Adding an adverse signal never moves an asset *into* NORMAL. If input x' differs from x only in making some predicate true, then $\sigma(x') \neq \text{NORMAL}$. *Proof.* By Property 1, NORMAL requires that predicate to be false. $\square$ This is the lattice's analogue of a tighten-only rule: every signal can only lower the permission level, and there is no input that can raise it except the conjunction of all predicates being false.

Property 3 (silence denies). If the reporter stops, then for every stock token there is a time $t^* = o + A$ after which S holds, so $\sigma = \text{STALE}$ and $\text{LTV} = 0$. The same holds for a silent feed ($t - u > h$) and a silent sequencer (Q). A liveness failure of any data source is therefore a *safety-preserving* failure. $\square$

Property 4 (portfolio default deny). Borrow capacity is evaluated over the whole portfolio:

$$\text{cap}(u) = \begin{cases} \sum_i v_i \cdot L_i / 10^4 & \text{if } \forall i : \sigma(a_i) = \text{NORMAL} \\ 0 & \text{otherwise} \end{cases}, \quad (4)$$

where v_i is the 18-decimal USD value of collateral i ; a borrow of Δ succeeds iff $\text{debt}(u) + \Delta \leq \text{cap}(u)$ and the vault holds the liquidity. One non-NORMAL asset zeroes the capacity of the entire position, not only its own contribution. A borrow that fails does not revert: it emits **BorrowBlocked** with the reason (**RISK_STATE**, **LTV_EXCEEDED**, **NO_COLLATERAL**, **INSUFFICIENT_LIQUIDITY**) and the blocking asset's state, which the interface explains in plain English (G6).

Property 5 (protected liquidation). A position is liquidatable iff $\text{debt}(u) > \sum_i v_i \cdot \min(L_i + 1500, 9500) / 10^4$, and the evaluation *reverts* with **LiquidationPaused(asset, state)** if any collateral is non-NORMAL. No user is liquidated on a halted, stale, deviating or corporate-action price (G4). Liquidations repay at most 50% of the debt (close factor) with a 5% bonus. $\square$

Configuration is bounded at write time: **setAssetConfig** rejects $h = 0$, $\delta = 0$, $\delta > 5000$ and $L_a > 8000$, so even the admin cannot configure a 100% LTV or a zero heartbeat.

6 Authenticated Market Status

Halts, corporate actions and reference prices do not exist in the price feed; they enter through signed reports. A report is the EIP-712 structure

```
MarketReport(address asset, bool halted, bool corporateActionPaused,
             uint256 uiMultiplier, uint256 referencePrice,
             uint64 observedAt, uint64 nonce)
```

under the domain (name "BloomRiskEngine", version "1", chainId, verifyingContract). Let (n', o') be the last accepted nonce and observation time for the asset. `submitReport` accepts a report iff

$$n > n' \wedge o \leq t \wedge t - o \leq A \wedge o \geq o' \wedge \rho \leq P_{\max} \wedge \text{lowS}(\text{sig}) \wedge \text{ecrecover}(D, \text{sig}) \in \mathcal{R}, \quad (5)$$

where D is the typed-data digest and $\mathcal{R}$ the reporter allowlist. Binding the domain to `chainId` and `verifyingContract` defeats cross-chain and cross-deployment replay; the strict nonce defeats in-chain replay; the order and age bounds prevent a reporter from backdating or resurrecting an old “all clear”; the low-s check (with $v \in \{27, 28\}$ and $r, s \neq 0$) removes signature malleability; and failures surface as typed errors (`ReplayedNonce`, `ReportFromFuture`, `StaleReport`, `OutOfOrderReport`, `InvalidSignature`, `UnauthorizedReporter`). Anyone may call the permissionless `refresh(asset)` to persist a state transition and emit its event; no one can change the inputs through it.

The reporter derives each report from the Robinhood API’s quote endpoint (reference price) and asset status. On testnet it also writes the live quote into a mock price feed before reporting, so that the whole path, from the API to the reporter, the signature, the transaction and the Stylus engine, runs on real data even though the tokens are mocks.

7 Consensus-Enforced Agent Authority

7.1 From a sentence to a policy

“Save \$500 for my laptop by December 15” is parsed deterministically (no model) into a typed goal: target, deadline, per-transaction cap, daily cap, maximum stock allocation and an asset allowlist; a goal with no date receives a bounded 90-day deadline, because the deadline is also the session key’s expiry. The optional model is consulted only for phrasing the parser does not recognize, and its output must pass the same schema validator, which rejects raw addresses, calldata and malformed amounts; recipients resolve only from the user’s contacts and assets only from the onchain registry. The user signs `createGoal` and `activateGoal` from their own wallet; the goal is then an onchain object in `BloomPolicy`.

7.2 The authorization predicate

For an agent call with target τ , value ν and calldata c (selector φ) on behalf of account α under goal g , `BloomPolicy` computes

$$\begin{aligned} \text{allow} \Leftrightarrow & \text{active}(g) \wedge t_{\text{create}} \leq t \leq t_{\text{deadline}} \wedge \nu = 0 \wedge (\tau, \varphi) \in \mathcal{K} \\ & \wedge \text{asset}(c) \in \text{Allow}_g \wedge \sigma(\text{asset}(c)) = \text{NORMAL} \wedge \text{usd}(c) \leq \text{perTx}_g \wedge \text{spent}_g(\text{day}) + \text{usd}(c) \leq \text{daily}_g^{(6)}, \end{aligned}$$

where $\mathcal{K}$ is a *closed* set: exact, capped `approve` of an allowed asset to the vault, router or claims contract only; `BloomVault.deposit` with the receiver fixed to the account itself; `StockRouter.send` and `swap`; and `BloomClaims.createClaim`. Every conjunct that fails yields a named reason (`NO_ACTIVE_POLICY`, `POLICY_EXPIRED`, `NATIVE_VALUE_NOT_ALLOWED`, `TARGET_NOT_ALLOWED`, `SELECTOR_NOT_ALLOWED`, `ASSET_NOT_ALLOWED`, `ASSET_RISK_BLOCKED`, `PER_TX_CAP_EXCEEDED`,

DAILY_CAP_EXCEEDED, RECEIVER_NOT_ALLOWED, SPENDER_NOT_ALLOWED, MALFORMED_CALLDATA, ...) that the chat renders verbatim in plain English. After a swap, the account additionally checks the goal’s stock allocation ceiling and reverts the whole action if it would be exceeded.

At the account layer, `validateUserOp` accepts the owner’s signature for anything; a session key’s signature is accepted only if the operation’s calldata is `executeByAgent(signer, ...)` with the signer as its own first argument, and the validity window returned to the EntryPoint is the goal’s own window, so expiry is enforced by the EntryPoint itself.

7.3 Why this is stronger than a firewall

Runtime guardrails for agents, including deterministic-first firewalls, sit in the application process: they are strong against a misbehaving model and weak against a compromised host, because the host is the enforcer. In Bloom the enforcer is the chain. The model is upstream of every check in Equation 6, the backend only relays, and the policy lives in contract storage that only the account owner can change. Formally, the set of calls the agent can execute is the intersection

$$\text{Auth}(g) = \mathcal{K} \cap \text{Allow}_g \cap \{a : \sigma(a) = \text{NORMAL}\} \cap \text{Caps}_g \cap \text{Window}_g, \quad (7)$$

and every off-chain component can only choose *within* it. This gives G2 by construction rather than by review: there is no code path, on or off chain, through which a model verdict or a server configuration widens Equation 7.

7.4 Authority analysis

Component	Move funds	Exceed goal	Mark NORMAL	Mark non-NORMAL	Change config	Liquidate on halt
Language model	—	—	—	—	—	—
Deterministic parser	—	—	—	—	—	—
Backend (agent key)	bounded	—	—	—	—	—
Agent session key	bounded	—	—	—	—	—
Reporter key	—	—	yes ^a	yes	—	—
Mock oracle (testnet)	—	—	yes ^b	yes	—	—
Guardian	—	—	—	—	—	—
Admin (multisig)	—	—	yes ^c	yes	yes	—
Account owner	yes	yes ^d	—	—	—	—
Any caller	—	—	—	—	—	—

Table 2: Authority analysis. Each row is a key or component; each column an outcome it could cause. “bounded” means only inside an active goal’s Equation 7. ^aOnly by signing a fresh, in-order report, and only if the price, freshness, deviation, token and sequencer predicates also hold. ^bTestnet only: mock feeds and token flags are writable by the mock-oracle key and never deployed on mainnet. ^cIndirectly, by reconfiguring bounds or the reporter set, within the write-time limits. ^dBy creating or revoking their own goals. The final column is empty by construction: `_liquidationValue` reverts before any seizure while any collateral is non-NORMAL.

8 Identity, Keys and the Service Boundary

The backend never trusts an address it is sent. A wallet signs an EIP-712 `BloomLogin` challenge (wallet, app, URI, chain, single-use nonce, 5-minute expiry); the session token is random and stored only as its SHA-256 hash, and every account endpoint acts for the session’s wallet, with a mismatched owner or recipient rejected with 403. Owner actions return as sign requests for the user’s wallet; risk

simulations require the onchain `DEFAULT_ADMIN_ROLE` on the vault; the testnet faucet is authenticated, chain-restricted and capped. Every role (deployer, admin, reporter, agent, claim authority, faucet, mock oracle) holds a distinct key; the service refuses to start if two roles share one, and the deployment script proves onchain that the deployer holds no role after handover. Ownership uses `Ownable2Step`, so a multisig admin’s acceptance is explicit.

9 Implementation

Stylus engine. Rust 1.91.0, `stylus-sdk` 0.10.6, `cargo-stylus` 0.10.9; the pure classifier (`risk.rs`, `eip712.rs`) has no VM dependency and is unit-tested natively. The deployed program is 32,795 bytes of compressed WASM stored in two fragments (activation data fee 0.000187 ETH), at `0xc464...b124` (deployment transaction `0xac69...e403`). The EVM twin, compiled with Solidity 0.8.28 (`viaIR`, Cancun), is 8,931 bytes of runtime bytecode and exposes the identical ABI, so every consumer is implementation-agnostic.

Protocol contracts. `BloomVault` (ERC-4626 with a virtual-share offset of 10^6 against first-depositor inflation, balance-delta checks rejecting fee-on-transfer tokens, `ReentrancyGuard` on every state change); `StockRouter` (canonical-only sends and swaps with non-zero minimum output and deadlines); `BloomClaims` (claim links authorized by an EIP-712 `ClaimAuthorization`); `BloomPolicy`; `BloomAccount` and its `CREATE2` factory; and `BloomAssetRegistry`, which accepts assets by exact address and checks decimals at registration, ignoring symbols and names.

Service. A TypeScript/Express backend with the in-process halt-aware reporter; a Next.js frontend with `RainbowKit` and `wagmi`; containerized for hosting with a liveness probe that does not depend on the RPC.

10 Evaluation

All measurements below were taken on September 25, 2026 against the live Robinhood Chain Testnet deployment or by the test suites at tag `bloom-v2-rc1`.

10.1 Correctness and parity

The risk specification is a hand-written vector file of 43 classification cases (every state, every boundary: equal to heartbeat versus one second past it, deviation exactly at the bound versus one unit over, 8-, 18- and 20-decimal feeds, grace-period edges, multiplier mismatch) plus an EIP-712 digest-and-signature vector. The Solidity library and the Rust crate are each tested against the same file; the parity goal G5 is therefore a checked property, not a porting claim.

Suite	Result	Scope
Hardhat	139 / 139	engine vectors onchain, vault, router, claims, policy, account, ERC-4337, role separation, demo flow
Foundry	11 / 11	property fuzzing (1,000 runs per property), vault invariants (128 runs $\times$ depth 64)
Stylus (Rust)	16 / 16	shared vectors, EIP-712 parity, contract behaviour
Reporter	9 / 9	report derivation, signing, scenario handling
Backend	22 / 22	intents, sign-in and replay, authorization matrix, rate limits, CORS, key separation, bounded waits
Demo, live testnet	13 / 13	sign-in lockdown, deposit, goal, agent send, halt and reset, claim link, halted-asset refusal
Wallet flow, live testnet	7 / 7	a fresh wallet signs every owner action; the backend never signs for it

Table 3: Automated verification. 197 unit and integration tests, plus two end-to-end suites run against the live testnet deployment and a local chain.

10.2 A live reporter cycle

To confirm the risk flow end to end we read the engine’s stored reports, let one reporter cycle run, and read them again. Every asset’s nonce advanced by exactly one (AAPL 70→71, NVDA 64→65, QQQ 69→70, SPY 65→66), the report ages fell to 58–90 s, and all four assets evaluated to NORMAL with a 60% maximum LTV. The AAPL report (transaction `0x57e3...dec0`, status 1, sent by the reporter key to the Stylus engine) carried a reference price of \$336.31; the Robinhood API quoted AAPL at a bid of \$336.14 and an ask of \$336.26 moments later. The halt scenario was then run through the same path: an admin-only simulation signs a `halted = true` report, the engine returns HALTED with LTV 0, borrowing and liquidation disabled, and an agent send of the halted asset is rejected onchain with `ASSET_RISK_BLOCKED`; a reset restores NORMAL.

10.3 Stylus versus EVM: a measured cost

Stylus is commonly presented as cheaper than the EVM. We measured it rather than assumed it. For the Stylus engine we take live receipts on Robinhood Chain Testnet and subtract the L1 data component that Arbitrum reports separately (`gasUsedForL1`); for `getRisk` we subtract `NodeInterface.gasEstimateL1Component` and the 21,000 intrinsic gas from `eth_estimateGas`. For the EVM twin we run the same operations on Hardhat (Cancun, `viaIR`) against the same mock feeds. Both measure a *steady-state* `submitReport`, updating an existing report, which is the reporter’s actual workload.

Operation	EVM twin	Stylus	Stylus / EVM
<code>submitReport</code> (median, execution gas)	100,811	158,613	1.57×
<code>getRisk</code> (execution gas)	≈ 55,500	≈ 108,200	1.95×
Runtime code size (bytes)	8,931	32,795 (compressed)	3.67×

Table 4: Measured with `scripts/bench-risk-engine.js` (no testnet transactions are sent). Stylus median over 12 live receipts, whose execution gas varied by 13 units; its L1 component ranged from 23,677 to 24,197 gas. EVM median over 5 runs. Hardhat’s schedule for EVM opcodes matches Arbitrum’s, but the two chains are not identical and the comparison is an estimate, not a controlled benchmark.

The result goes against the usual claim, and its cause is structural. `getRisk` performs almost no arithmetic: it makes four external static calls (the sequencer round, the feed round, and the token’s pause flag and multiplier; feed decimals are cached at configuration), and each is a cross-VM call from WASM into EVM contracts, with ABI encoding and host-I/O overhead on every call, plus the fixed cost

of entering a Stylus program. Stylus pays off for compute-dense logic (cryptography, numeric kernels, parsing), not for a thin coordinator over EVM oracles. We keep the Stylus deployment because the engine’s value is its *verifiability*: a memory-safe Rust core with no VM dependency, unit-tested natively against the same vectors as the Solidity twin. We do not claim a gas advantage, and Section 15 describes how the read path could close the gap.

10.4 Operating cost

Transaction (testnet, 0.01 gwei)	Gas used	ETH
Signed market report (Stylus)	168,075–182,810	$\approx 1.8 \times 10^{-6}$
Mock feed update (testnet only)	56,142	5.6×10^{-7}
Account creation (sponsored)	168,387	1.7×10^{-6}
Faucet mint (testnet only)	61,503	6.2×10^{-7}

Table 5: Per-transaction costs from receipts. At a 120-second cycle over four assets the reporter spends about 0.005 ETH per day (0.035 ETH per week); a 300-second cycle reduces this by 60%. The full testnet deployment cost about 0.00053 ETH. A fork dry-run of the gated mainnet deployment estimated 18 transactions and 13.35 M L2 gas, about \$1.50 at the prices of September 24, 2026, above the script’s default \$0.01 safety limit, so it refused to deploy.

10.5 Latency

The RPC health probe measured 36–1,017 ms round trips to the public Robinhood RPC over the day. A report cycle over four assets completed in about 20 s, dominated by sequential inclusion under the per-key lock. Individual reads measured 3.8 s for an account view, 5.6 s for goals and 8.8 s for activity, which scans 50,000 blocks of logs; under the reporter’s concurrent load, six parallel reads timed out at 25 s before read coalescing (F2 in Section 11), after which the home screen loaded in about 3 s.

11 Findings from Building Against Live Systems

Every integration was exercised against the running chain, RPC, hosting platform and wallets rather than their documentation. Each finding below would have shipped as a silent failure otherwise; each has a fix and, where it is a logic change, a regression test.

F1. The default HTTP transport hung. ethers v6’s built-in HTTP transport never resolved on Node 26; every component now uses a `fetch`-based transport installed at startup.

F2. The public RPC throttles bursts. A home screen issued six concurrent account reads while the reporter was sending transactions; all six timed out at 25 s. Identical in-flight reads for one wallet are now coalesced into one chain read (nothing is cached after it settles, so reads after a write stay fresh), and every RPC request carries a 20 s timeout mapped to a readable 503.

F3. An unbounded receipt wait froze a key. A stalled RPC held one agent request for more than five minutes, and because transactions are serialized per key, every later action for that key queued behind it. Receipt waits are now bounded (90 s server, 120 s client) and a timeout returns 504 *pending* with the transaction hash instead of failing or retrying blindly.

F4. Empty environment variables defeat defaults. A hosting dashboard stored an optional field as the empty string; `process.env.X ?? default` kept the empty string, and the service crashed with “no RPC URL”. Worse, `Number("")` is 0, so an empty `PORT`, `SESSION_TTL_SEC` or `REPORTER_INTERVAL_SEC` would silently have become zero. Empty variables are now treated as unset before anything reads them.

F5. A fresh chain reuses addresses. A restarted local node redeploys every contract to the same address, so a data store keyed by address resurrected a previous run’s faucet ledger. Local stores are now keyed by the deployment’s timestamp; testnet stores by chain.

F6. A default RPC pointed at the wrong chain. A smoke script’s RPC defaulted to the local node while testing testnet, and read testnet token addresses from a chain where they held no code, returning `0x`. Scripts now derive the RPC from the deployment and assert that its chain ID matches the backend’s.

F7. Request caching reuses nonces. ethers deduplicates identical RPC reads within 250 ms; on an instant-mining chain two back-to-back sends read the same transaction count and the second failed with “nonce already used”. Signing processes disable the cache.

F8. Idempotent role handover is not automatic. Re-running the handover to an admin that already held its roles would have made the deployer-side “renounce” step strip the admin of them. The handover now never renounces a role its target should hold, and a test runs it twice.

F9. Toolchain argument order matters. `cargo stylus deploy` consumed `--private-key-path` as a constructor argument when it followed the variadic `--constructor-args`, and required `--no-verify` without a container runtime. The deploy script fixes the order; source verification is reported as not yet done (Section 13).

F10. Wallets report “disconnected” while reconnecting. On page load the wallet library briefly reports *disconnected* before restoring the connection; treating that as a sign-out forced a new signature on every reload. Only a real connected→disconnected transition now ends a session.

F11. Error text is not a stable interface. A wallet with no gas produced “the total cost (gas × gas fee + value) of executing this transaction exceeds the balance of the account”, which never contains “insufficient funds”; users saw raw text. The mapping now matches both forms, verified with a zero-balance wallet.

F12. A retired key can outlive its rotation. After rotating every role off the original single deployer key, an audit of all goals found one (goal #1) still naming the retired key as its agent, created by a throwaway test wallet whose key no longer exists. No admin path can revoke it (Section 13); its exposure is bounded by its own policy: at most \$50 per day of mock USDG from one orphaned account, expiring December 15, 2026.

F13. Stylus is not cheaper for call-bound logic. Measured in Section 10.3, contrary to our initial expectation; reported rather than omitted.

12 Security Analysis

Adversary or failure	Outcome and mechanism
Prompt-injected or wrong model	Proposes an intent; the validator rejects raw addresses and calldata; Equation 6 rejects anything outside the goal.
Compromised backend	Holds only the agent key: bounded to active goals' Equation 7; cannot move owner funds, sign for users, or change risk state.
Stolen agent session key	Same bound as above until the goal is revoked or expires; the EntryPoint enforces expiry.
Replayed or reordered report	Rejected by the strict nonce, order and age bounds and the chain-and-contract domain (Equation 5).
Silent reporter, feed or sequencer	Assets become STALE or SEQUENCER_DOWN; LTV drops to 0 (Property 3).
Manipulated or diverging price	DEVIATION when the price leaves the signed reference by more than δ ; out-of-range prices are INVALID_PRICE.
Liquidation during a halt	LiquidationPaused revert while any collateral is non-NORMAL (Property 5).
ERC-4626 inflation, donation	Virtual-share offset 10^6 ; solvency invariant fuzzed.
Fee-on-transfer or rebasing token	Balance-delta checks reject the transfer.
Token spoofing (same symbol)	Registry, router and vault accept canonical addresses only.
Signature malleability	Low-s, non-zero $r, s, v \in \{27, 28\}$ in both implementations.
Impersonation at the API	EIP-712 sign-in; the session wallet overrides any supplied address.
Key reuse across roles	Service refuses to start; deployment proves the deployer holds no role.

Table 6: Adversaries and failures considered, with the mechanism that bounds each.

13 Trust Assumptions and Limitations

Stated plainly, in decreasing order of weight.

1. **A single reporter key is trusted for market status.** A compromised reporter can mark an asset halted (safe) or sign a false “all clear”. The latter is bounded, not prevented: the price must still be fresh, valid and within the deviation bound of the reporter’s own reference, the token must not be paused, and the sequencer must be up. A k-of-n reporter quorum is future work.
2. **Not audited.** This is a testnet release candidate.
3. **Testnet assets are mocks.** USDG, the four stock tokens and their feeds are testnet mocks relaying live Robinhood API quotes; the risk engine and all enforcement contracts are real. Robinhood Chain Testnet has no canonical USDG, Stock Tokens or Chainlink stock feeds.
4. **No admin-side agent revocation.** Only an account or its owner can revoke a goal. A compromised agent key remains usable inside the goals that name it until they are revoked or expire. F12 is a live instance; a guardian-controlled agent denylist in **BloomPolicy** is required before mainnet.
5. **Stylus costs more here, and its source is not yet verified.** Table 4; reproducible source verification requires a container runtime that was unavailable on the build host.
6. **Simplified lending economics.** Borrowing is interest-free in the prototype and USDG is valued at \$1; the mainnet USDG/USD feed is configured but not wired.
7. **No sequencer uptime feed exists for Robinhood Chain.** The mainnet configuration disables the sequencer guard explicitly and visibly onchain; freshness and report age still apply.
8. **Single-instance service.** Sign-in sessions, nonces and rate limits are held in memory; the public RPC is rate-limited. Wallet sign-in supports externally owned accounts only (no ERC-1271).

14 Related Work

Oracle-based lending. Money markets price collateral from Chainlink feeds with heartbeat-based staleness checks, and some add an L2 sequencer-uptime grace period (for example Aave v3’s price-oracle sentinel). Bloom adopts both and adds what equities need and crypto assets do not: halts, corporate actions and multiplier consistency, a signed reference for deviation, and portfolio-wide default deny with protected liquidation. **Tokenized equities.** Robinhood Stock Tokens and other tokenized-stock programs make US equities transferable onchain; the risk of lending against them around market hours is the gap Bloom addresses. **Agent wallets.** ERC-4337 accounts and modular session-key systems scope what a delegated key may do; Bloom’s contribution is to couple that scope to a live risk state and to derive it from a plain-English goal. **Agent guardrails.** Content guardrails govern conversations; runtime firewalls such as Vigil govern tool calls with deterministic-first pipelines in the application process. Bloom moves the enforcement point for financial side effects from the process to consensus, so it survives the compromise of the process itself. **Arbitrum Stylus** enables Rust contracts alongside the EVM; our measurement (Table 4) is one data point on where its cost model favours WASM and where it does not.

15 Future Work

A k-of-n reporter quorum with independent data sources; a guardian-controlled agent denylist and emergency revocation in **BloomPolicy**; closing the Stylus cost gap by batching the engine’s oracle reads (one cross-VM call per dependency per block) and by moving the vector-checked classifier into a pure Stylus library called from EVM consumers; reproducible Stylus source verification; an external audit; ERC-1271 sign-in for smart-contract wallets; an interest-rate model and the USDG/USD feed; and a mainnet deployment against canonical Stock Tokens and Chainlink stock feeds, which the gated deployment script already supports behind its cost limit.

16 Conclusion

For a tokenized stock, a fresh price is not a permission. Bloom makes the difference explicit and enforceable: a default-deny lattice in which the only permitting state requires every adverse signal to be absent and silence always denies; market status that arrives signed, in order and bounded in age; and agent authority that is the intersection of the user’s onchain policy and the live risk state, enforced by the chain, so neither a model nor a compromised server can widen it. We built it against live systems and report what they told us, including thirteen defects the documentation did not predict and a Stylus measurement that contradicts the usual pitch.

Bloom doesn’t replace the price oracle. It adds equity-specific risk interpretation around it.

References

- [1] Robinhood. *Robinhood Chain documentation* and *Stock Token asset API*. docs.robinhood.com; api.robinhood.com/rhj, 2026.
- [2] Offchain Labs. *Arbitrum Stylus* and *NodeInterface* documentation. docs.arbitrum.io, 2026.
- [3] Chainlink. *Data Feeds*, *L2 Sequencer Uptime Feeds* and *Robinhood Chain feed directory*. docs.chain.link, 2026.
- [4] Bloemen, R., Logvinov, L., Evans, J. *EIP-712: Typed structured data hashing and signing*. Ethereum Improvement Proposals, 2017.

- [5] Buterin, V. et al. *ERC-4337: Account Abstraction Using Alt Mempool*. Ethereum Improvement Proposals, 2021.
- [6] Santoro, J. et al. *ERC-4626: Tokenized Vaults*. Ethereum Improvement Proposals, 2021.
- [7] Aave. *Aave v3 technical paper* (price oracle sentinel). aave.com, 2022.
- [8] OpenZeppelin. *Contracts 5.x*: ERC-4626, AccessControl, Ownable2Step, ECDSA, ReentrancyGuard. openzeppelin.com.
- [9] Project Vigil. *A Deterministic-First Runtime Firewall for Autonomous AI Agents*. github.com/Aaditya1273/Vigil, 2026.
- [10] Project Bloom. *Source, deployment manifests, test vectors and benchmark*. github.com/Aaditya1273/BLOOM, 2026.

Bloom is hackathon software and has not been audited. Measurements in this paper were taken on September 25, 2026 against Robinhood Chain Testnet (chain 46630) and by the suites named in the text at tag `bloom-v2-rc1`; the gas comparison is reproducible with `scripts/bench-risk-engine.js`, the test vectors are in `test/vectors/risk-vectors.json`, and the deployment is in `deployments/robinhood-testnet.json`. Testnet assets are labelled mocks. Nothing in this paper claims a result that was not measured.